Щучкин Максим, Б07-202

# Отчет по лабораторной работе #6

## «Параллельная сортировка распределенного массива»

**Цель:** реализовать решение задачи сортировки, написав функцию слияния и подобрав необходимое число циклов обменов. Протестировать на размерностях массива 10000-1000000.

**Текст программы:**

```
#include <stdio.h>
#include <stdlib.h>
#include <math.h>
#include <mpi.h>
#include <time.h>

/* компаратор для qsort: возвращает -1, 0 или 1 без риска переполнения */
int cmp_double(const void *a, const void *b) {
    double da = *(const double*)a;
    double db = *(const double*)b;
    return (da > db) - (da < db);
}

int main(int argc, char *argv[]) {
    MPI_Init(&argc, &argv);
    int np, mp;
    MPI_Comm_size(MPI_COMM_WORLD, &np);
    MPI_Comm_rank(MPI_COMM_WORLD, &mp);

    int ns = (argc > 1) ? atoi(argv[1]) : 10000;
    if (ns < np) ns = np;

    /* Равномерное распределение */
    int base_nc = ns / np;
    int rem = ns % np;
    int nc = base_nc + (mp == np - 1 ? rem : 0);

    /* буфер */
    double *a1 = (double*)malloc(nc * sizeof(double));
    if (!a1) { perror("malloc"); MPI_Abort(MPI_COMM_WORLD, 1); }

    /* Генерация случайных чисел (разные seed для каждого процесса) */
    srand(time(NULL) + mp);
    for (int i = 0; i < nc; i++) a1[i] = (double)rand() / RAND_MAX * 32768.0;

    double t1 = MPI_Wtime();

    /* Локальная сортировка каждого фрагмента */
```

```
    qsort(a1, nc, sizeof(double), cmp_double);

    /* Буферы для обмена и временного слияния */
    int max_nc = base_nc + (rem > 0 ? 1 : 0);
    double *a2 = (double*)malloc(max_nc * sizeof(double));
    double *tmp = (double*)malloc((nc + max_nc) * sizeof(double));

    MPI_Status status;
    /* Четно-нечетная перестановочная сортировка: np фаз гарантируют
глобальную упорядоченность */
    for (int phase = 0; phase < np; phase++) {
        int partner = -1;
        if ((phase + mp) % 2 == 0 && mp + 1 < np) partner = mp + 1;
        else if ((phase + mp) % 2 != 0 && mp - 1 >= 0) partner = mp - 1;

        if (partner != -1) {
            int partner_nc = base_nc + (partner == np - 1 ? rem : 0);

            /* Атомарный обмен с соседом */
            MPI_Sendrecv(a1, nc, MPI_DOUBLE, partner, 0,
                         a2, max_nc, MPI_DOUBLE, partner, 0,
                         MPI_COMM_WORLD, &status);

            int recv_cnt;
            MPI_Get_count(&status, MPI_DOUBLE, &recv_cnt);

            /* Слияние двух отсортированных массивов во временный буфер */
            int i = 0, j = 0, k = 0;
            while (i < nc && j < recv_cnt) {
                tmp[k++] = (a1[i] < a2[j]) ? a1[i++] : a2[j++];
            }
            while (i < nc) tmp[k++] = a1[i++];
            while (j < recv_cnt) tmp[k++] = a2[j++];

            /* Распределение: процесс с меньшим рангом забирает меньшие
элементы */
            if (mp < partner) {
                for (int x = 0; x < nc; x++) a1[x] = tmp[x];
            } else {
                int start = recv_cnt;
                for (int x = 0; x < nc; x++) a1[x] = tmp[start + x];
            }
        }
    }

    double t2 = MPI_Wtime();
    double amin = a1[0];
    double amax = a1[nc - 1];
```

```
    fprintf(stderr, "mp=%d ns=%d i1=%d i2=%d nc=%d amin=%le amax=%le
t1=%le\n",
            mp, ns, 0, ns-1, nc, amin, amax, t2 - t1);

    free(a1); free(a2); free(tmp);
    MPI_Finalize();
    return 0;
}
```

**Output:**

```
fnbik09@sr210:~/mpc/task6$ mpicc -O2 solution_06.c -o solution_06.px -lm
fnbik09@sr210:~/mpc/task6$ mpirun -np 1 solution_06.px 10000 2>&1 | grep -v Authorization
mp=0 ns=10000 i1=0 i2=9999 nc=10000 amin=7.341522e+00 amax=3.276392e+04 t1=1.319839e-03
fnbik09@sr210:~/mpc/task6$ mpirun -np 2 solution_06.px 100000 2>&1 | grep -v Authorization
mp=0 ns=100000 i1=0 i2=99999 nc=50000 amin=2.129517e-01 amax=1.641227e+04 t1=1.490510e-02
mp=1 ns=100000 i1=0 i2=99999 nc=50000 amin=1.641287e+04 amax=3.276772e+04 t1=1.428435e-02
fnbik09@sr210:~/mpc/task6$ mpirun -np 4 solution_06.px 1000000 2>&1 | grep -v Authorization
mp=0 ns=1000000 i1=0 i2=999999 nc=250000 amin=3.295898e-03 amax=8.215441e+03 t1=7.896711e-02
mp=3 ns=1000000 i1=0 i2=999999 nc=250000 amin=2.457863e+04 amax=3.276800e+04 t1=7.624524e-02
mp=1 ns=1000000 i1=0 i2=999999 nc=250000 amin=8.215509e+03 amax=1.640177e+04 t1=7.740304e-02
mp=2 ns=1000000 i1=0 i2=999999 nc=250000 amin=1.640178e+04 amax=2.457859e+04 t1=7.358501e-02
```

**Выводы:**

1) Диапазоны значений на процессах строго возрастают (amax процесса k ~ min процесса k+1), что подтверждает правильную работу алгоритма слияния и фазовых обменов.
2) Время выполнения t1 определяется в основном локальной сортировкой, а накладные расходы на np фаз синхронных обменов остаются малыми - эффективно.
3) Использование MPI_Sendrecv полностью исключило дедлоки при одновременных обменах, а условие mp < partner гарантировало корректное распределение элементов без дополнительных барьеров.